

ON BUILDING

SYSTEMS

Prompt LLMs Reliably



#ASLI ENGINEERING

BY

ARPIT BHAYANI

Welcome! Get the best out of this course

1. 3 week course (intense & focus)
2. be hands-on w/ assignments, prototypes, reading material
3. read the pre-reads ↳ take your time
4. post reads and exercises are optional



Arpit Bhayani 
@arpit_bhayani

...

Everyone talks about agent intelligence. Then agents reach production - or face even the slightest bit of load - and suddenly the hard problems are:

- memory management
- concurrency
- backpressure
- retries
- timeouts
- failure handling
- observability

Agentic loops are long-running, non-deterministic, and inherently error-prone, which makes reliability difficult. The more complex the agents, the more interesting the failure modes.

Turns out, agents are often more of a classic software engineering problem than an AI problem. Good engineering practices are not going anywhere.

7:10 PM · May 30, 2026 · **11.6K** Views



24



14



335



148



Agenda

1. LLMs are stochastic	6 mins	
2. CoT v/s Few shot v/s Direct	12 mins	
3. Prompt Failure Modes	7 mins	
		10mins QnA
4. Structured Outputs	5 mins	
5. Prompting Reliably	10 mins	
		10min QnA

Prototypes: 6 (60 mins)

System: Fact-checker (using model's internal data) 40 mins

accuracy / correctness, etc

CPU hardware, FP operations,
Batching, Quantization

affects variance not intelligence

explores the tail

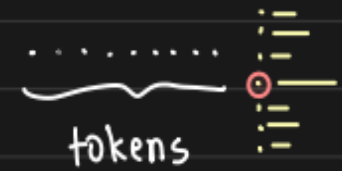
Temperature - 0 to 1

greedy decoding
(always the one w/
highest probability)

picks token w/
highest probability and
repeats

LLMs are stochastic programs

Hence are non-deterministic



Hence,

1. no 100% stable across re-runs with same input
2. accuracy variations

Treat LLM output non-reproducible

Prototype : Non-determinism

↳ sentiment analysis on a review

unconstrained

constrained

the ultimate goal

Shift the probability mass toward correct answers

Chain of Thought Prompting and Few Shot Prompting

②

adds intermediate reasoning
steps before the final answer
zero-shot cues

- let's think step by step

Few shot exemplars

- Problems with fully
worked out reasoning chains

①

prepend solved examples to the input

"model shifts distribution toward
structurally similar outputs"

↑
in-context learning

Quality matters

ARPIT BHAYANI

zero shot CoT: fast to implement, no example curation,

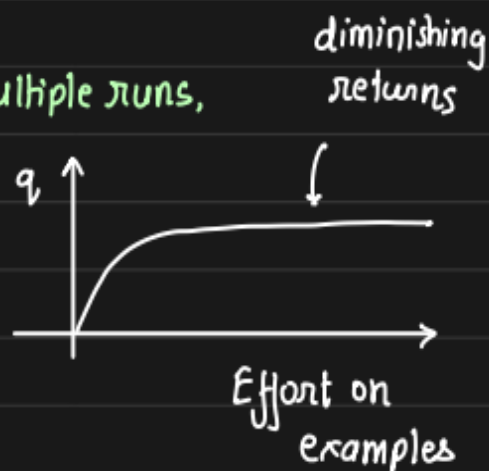
reasoning structure may vary

Few shot CoT: stable reasoning structure across multiple runs,

slower, more token consumption

* zero-shot CoT on modern LLM often sufficient

↓
most models like o1, o3, deepseek
internalizes the reasoning chain



Prototype: CoT v/s Direct

↳ 5 reasoning questions requiring multi-step reasoning

8 runs for each Direct: 32/40

CoT: 40/40 (zero shot)

Prototype: Direct v/s Few shot CoT v/s zero-shot CoT

↳ 5 reasoning questions requiring multi-step reasoning

Direct: 4/5 213 Tokens

Zero shot: 5/5 1089 Tokens

Few shot: 5/5 2024 Tokens

if Few shot not giving 'accurate', you can always
iterate multiple times and do a majority voting

trying out multiple
reasoning path a model
might take & picking
majority.

cost, time, accuracy

self-consistency

ARPIT BHAYANI

Prompt Failure Modes

prompt working fine started 'failing'

1. Hallucination → output contradicts info in the context (catchable)
↳ model fabricates info not in the context (hard to catch)
2. Prompt Injection → we discuss this in week 3 at length
3. Sycophancy - model changing correct answer when user pushes back
ways to handle this...
 - math gaslight
 - emotional blackmail
 - physical impossibility
 - guilt trip
 1. "Do not validate assumptions before checking"
 2. "if reasoning is wrong, tell me directly"
 3. "prioritize accuracy over user satisfaction"
 4. once you get the answer...
 - push back and answer shouldn't change
 - ask model to rate its own confidence

Prototype: Sycophancy - 5 questions 1 push back

Prototype: Prompt Injection

1. vulnerable
2. defence: input sanitization → int, string, format, city name, etc & "ignore all previous instruction"
3. defence: LLM classifier → make another model check before exec
4. defence: Sandbox → XML delimiters

(breaks parser)

- confidence v/s confidence-score
- model may miss fields it finds irrelevant
- prefix: "Here's the json data:"

Structured Outputs

→ LLMs are just Next token predictors

Free form text is natural output of model

But downstream systems need 'specific information'

Aug 2024 onwards structured outputs is a solved problem

→ write a parser → **LLGuidance**

→ structured output - JSON

→ earlier loop and hope (↑ latencies)

Production gotchas (JSON output)

1. deep nesting confuses models. keep schemas flat
2. enums are safe, use them a lot (wherever applicable)
3. token overhead - 50-300 per call (given how json is as a format)

Doc intelligence, agent-agent commⁿ
content moderation pipeline

even YYYY-MM-DD or [0-9]{8} works

Rule of Thumb

schema constrained json

single classification from fixed set → **enums**

structured data with multiple fields → **Pydantic / Zod**

valid SQL as output → **LLGuidance** - enforces arbitrary CFG on LLM output

Try this out

(grammar constrained generation)

it works with providers or runtimes that exposes token-level generation ctrl.

LLGuidance needs access to logits, token stream, sampling loop, next token selⁿ

ex: vLLM, llama.cpp, self hosted inference servers

* Regression testing against structured output is essential

lack of confidence, schema mismatch

ARPIT BHAYANI

Prompting LLMs Reliably

Most engineers treat prompt as quick note to LLM.

Review this python function

Review the following python function for correctness.... Do not comment on

style.... Return findings as numbered list. Each item should name the issue and also suggest a fix. If no issues found return 'no issues found'

task, constraint, output

↓
length, tone

↓
json, list, para ...

Explicit v/s Implicit

↓
For creative task do not specify
↳ tags, topics, itinerary, ...

→ When one of X matters
categories, attributes (json), etc

* Neither overspecify, nor underspecify

Positive v/s Negative Instructions

Do not be verbose

Respond in no more than 100 words

Do not make assumptions

if info missing, ask clarifying questions

Do not hallucinate

if you are unsure, say so explicitly

Monolithic Prompts

... extract ... do ... filter ...

... summarize, ...

Handle Edge cases in prompts

if ... HAPPY PATH ...

if no return 'no issues found'

Prompt is 'code'

version it

write evals

Step 1: _____

Step 2: _____

Step 3: _____

* Do not over do this ... it is not code

Strike a balance

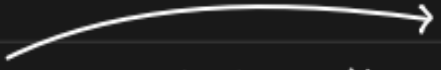
Judgement / Taste

Evals in subsequent sessions

ARPIT BHAYANI

* citations - when LLM responds

Fact checking Agent (Pipeline)

Given an input passage, fact check it.  stream of it

Use model's internal data + Majority Voting

* correctness >>> even under partial failure and non-determinism

input: English article, 5-30 verifiable factual claims

Checking fact $\Rightarrow$ model's own knowledge

Extension: websearch + majority (pre-req to be covered in Session 2)

1 article per minute w/ 20 claims

Peak load 10 articles

Output: SUPPORTED, REFUTED, UNVERIFIABLE

confidence score $[0,1]$, a short justification, source votes

Practical Use cases

1. Medical - symptom $\rightarrow$ urgent / non-urgent (best of 5)
2. Code Bug Severity Rating
3. Sentiment on ambiguous text
4. Data Labeling and Classification
5. Content moderation
6. Financial Data Extraction (extracting revenue)

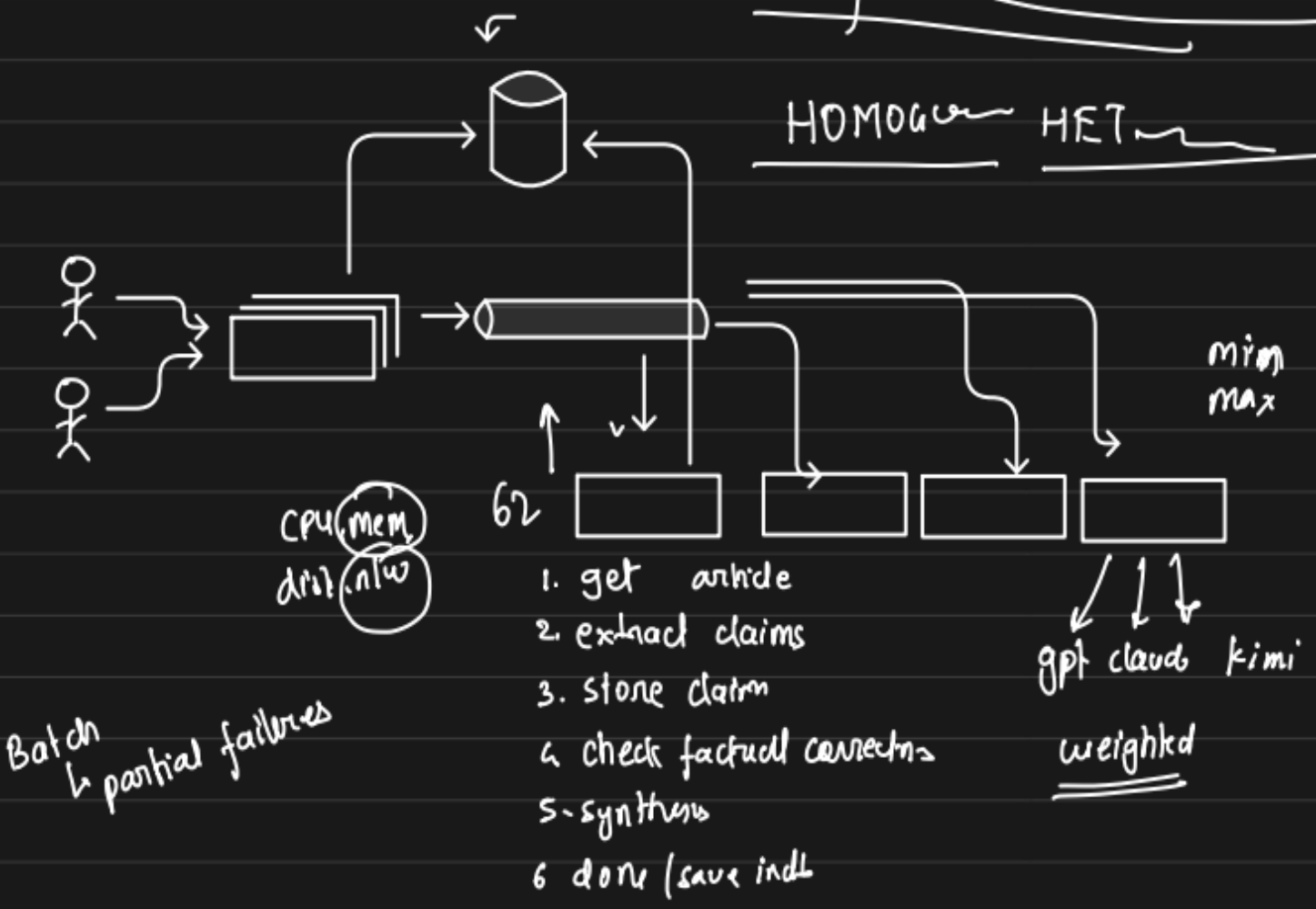
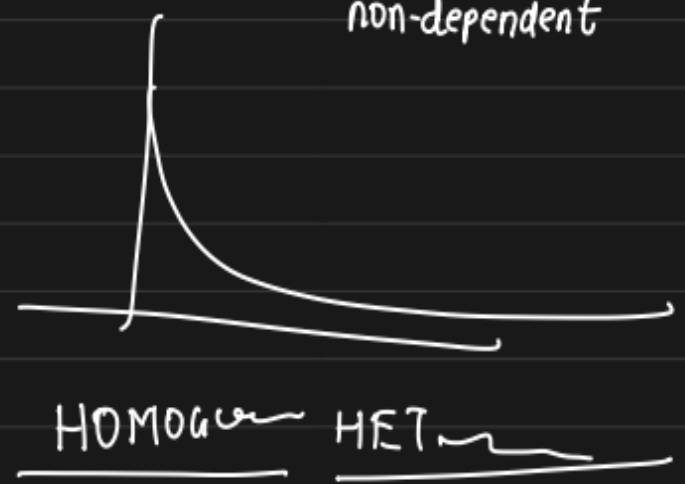
Solutioning....

- ✓ 1. Functional and Non-Functional Requirements
- ✓ 2. Number crunching - LLM calls per article
- ✓ 3. Capacity Estimations - Storage
- ✓ 4. High level Design and Pipeline
5. Agentic Loop Pseudocode
6. Non functional guarantees and pessimism
7. Prompts

Brainstorm

- 1. Fact checking...
- 2. Correctness >>
- what is a factual claim?
 - ↳ not an opinion

x multipass $\xrightarrow{3x}$ 20 claims $\xrightarrow{\times 10}$ 200 claims $\rightarrow$ batch API call
 latency < 10sec
 cost as min as possible
 availability
 threads!!
 not 4m 24hr on $\rightarrow$ one batch API call
 non-duplicate claims
 batch, time non-dependent



Functional Requirements:

1. identify atomic checkable claims (not opinions)

2. low confidence score = UNVERIFIABLE

3. idempotent per article hash

↳ identical output

4. No starvation

→ X is beautiful

Non Functional Requirements

1. not latency critical

2. 1 article per minute w/ 20 claims

3. Peak load 10 articles

4. p50 of 20 claim article < 90 seconds

p90 of 20 claim article < 3 minutes

} dominated by LLM inference time

3 sample × 20 claim = 60 LLM calls

3 way parallelism → $60/3 \approx 20-40$ seconds

* capacity pressure is entirely LLM.

LLM calls per article = $1_{\text{extraction}} + N_{\text{claims}} \times k_{\text{samples}} + 1_{\text{synthesis}}$

For $N=20$, $k=3$ LLM calls = $1 + 3 \times 20 + 1 = 62$ calls

For 10 concurrent articles, 620 calls

Storage

Article metadata = 500B

Claim: $N \times 1\text{KB} \rightarrow \text{text, votes, justification}$ $20 \times 1 = 20\text{KB}$

Raw LLM response

K response per claim $\sim 2\text{KB}$ each

$60 \times 2 = 120\text{KB}$

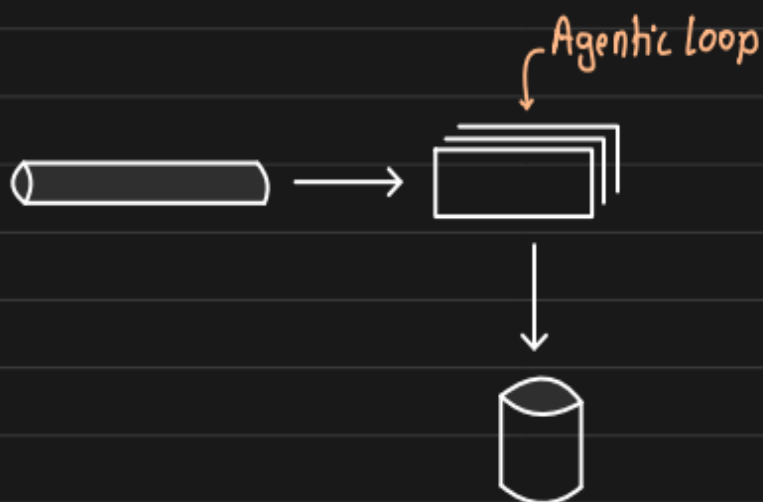
Total per article = 140KB

1000 articles per day = $140\text{KB} \times 1000 = \underline{\underline{140\text{MB}}}$ per day

Pipeline

Pipeline has 4 logical components

1. Article Ingestion
2. Claim Extraction
3. Verification by Voting
4. Synthesis



Wang et al. (2022) Majority voting + self consistency

→ more than this
incoherent

* correct reasoning paths converge with $\tau > 0$ [0.5, 0.8]

while hallucinations scatter

↓
less than this... Converge

Agentic loop

```
article_id = hash(article_text)
```

```
if result_store.get(article_id).status == DONE:
```

```
    return result_store.get(article_id).report // idempotency
```

```
claims = claim_extractor.run(article_text)
```

```
result_store.save(article_id, claims, status=VERIFYING)
```

```
verification_tasks = []
```

```
for claim in claims:
```

```
    for sample_index in range(K):
```

```
        verification_tasks.append((claim, sample_index))
```

```
// Run in parallel batches of BATCH_SIZE
```

```
results = parallel_execute(verify_claim, verification_tasks, batch_size=BATCH_SIZE)
```

```
verdicts = vote_aggregator.aggregate(results)
```

```
result_store.save(article_id, verdicts, status=AGGREGATING)
```

```
report = report_synthesizer.run(article_text, verdicts)
```

```
result_store.save(article_id, report, status=DONE)
```

```
return report
```

char offset

→ claim.id, text, char span

no shared conv history,

→ independent ($\tau > 0$)

→ list of verdicts

+ justification

→ Handle LLM rate limiting

with retries with exp backoff

Produce human readable summary paragraph

that contextualizes the findings

Pessimism

1. state management → checkpoint and resume, make verification idempotent

2. backpressure → 429, exp backoff with jitter

3. failure mode → default unverifiable, OR

more than 60% done → show confidence and pick final verdict

Prompts

Claim Extraction

```
You are a precise fact-checking assistant.

Given the following article, extract every atomic, verifiable factual claim.

Rules:
- Include only objective, checkable facts (names, dates, numbers, events, attributions)
- Exclude opinions, predictions, rhetorical questions, and tautologies
- Each claim must be self-contained -- do not use pronouns that refer outside the claim
- Return ONLY a JSON array, no preamble, no markdown fences

Schema:
[
  {
    "claim_id": "<uuid>",
    "text": "<the claim as a standalone sentence>",
    "span_start": <character offset in original article>,
    "span_end": <character offset in original article>
  }
]

Article:
{{article_text}}
```

```
You are a fact-checking assistant with broad world knowledge.

Evaluate the following claim and return a verdict.

Claim: {{claim_text}}

Verdicts:
- SUPPORTED: the claim is accurate based on your knowledge
- REFUTED: the claim is demonstrably false based on your knowledge
- UNVERIFIABLE: you lack sufficient knowledge to evaluate the claim, or it is ambiguous

Rules:
- Do not hedge -- pick the single best verdict
- If you are uncertain, prefer UNVERIFIABLE over a low-confidence SUPPORTED or REFUTED
- Return ONLY valid JSON, no preamble, no markdown fences

Schema:
{
  "verdict": "SUPPORTED" | "REFUTED" | "UNVERIFIABLE",
  "confidence": <float 0.0 to 1.0>,
  "reasoning": "<one to three sentences explaining your verdict>"
}
```

Verification

```
You are a fact-checking editor writing a summary of findings.

Below is an article followed by the verified verdicts for each factual claim extracted from it.
Your job is to write a 2-4 sentence summary paragraph for a reader who wants to know:
- How many claims were checked
- How many were supported, refuted, or unverifiable
- Which refuted or unverifiable claims are most significant

Rules:
- Do not re-adjudicate any verdict -- treat all verdicts as final
- Do not quote the article verbatim
- Be concise and neutral in tone
- Return only the summary paragraph, no preamble

Article:
{{article_text}}

Verdicts:
{{verdicts_json}}
```

Report Synthesis